

Parcial resuelto Teoría de Algoritmos

Fuerza Bruta: El problema de coloreo de grafos intenta asignar un color a cada nodo de forma tal que ningún par de nodos adyacentes comparten el mismo color. Sea el grafo $G=(V,E)$ se desea obtener una coloración de no más de k colores. Resuelva utilizando Backtracking.

Greedy: La costa de un país cuenta con “ n ” faros. Cada faro “ i ” cubre una cantidad de kilómetros de costa lineal no interrumpida a su alrededor comenzando por el kilómetro s_i y finalizando en el kilómetro e_i . Muchos de esos faros se solapan en su cobertura, incluso algunos están totalmente cubiertos entre sí. Para minimizar la asignación de personal que requieren nos solicitan que seleccionemos el menor subconjunto de faros que cubran totalmente la línea costera o afirmar que no es posible. Proponga un algoritmo greedy que lo resuelva.

Programación dinámica: El dueño de un salón de convenciones debe decidir a qué exposiciones y congresos alquilará en los próximos meses. Ha reunido “ n ” propuestas cada una de ellas con una ganancia diferente según los servicios solicitados. Algunos de ellos se superponen entre sí. Para cada propuesta conoce: fecha de inicio, fecha de finalización, monto a ganar por realizarlo. Planea elegir las que duren menos. Mostrar que esta solución no es óptima. Proponer una solución utilizando programación dinámica que responda propuestas elegir y cuánto se puede ganar.

Redes de Flujo: Un museo estrena un nuevo sistema de seguridad con “ n ” cámaras. En una nueva exposición, siguiendo las indicaciones del curador, se ubicaron “ m ” piezas. Cada cámara solo puede cubrir una pieza. Por las ubicaciones se conoce que cámaras pueden cubrir qué piezas. Resolver el problema utilizando redes flujo respondiendo si es posible cubrir todas las piezas y en ese caso qué asignaciones realizar.

Clases de Complejidad: Para elaborar un juego de aventura gráfica se juntaron “ m ” equipos de diseñadores para proponer diferentes escenarios. No todos los escenarios son compatibles entre sí. Por cada una de las “ n ” escenarios han anotado que equipo lo propuso y con cuales otros no es compatible. Desean poder seleccionar “ k ” escenarios compatibles para construir el juego. Se requiere que todos los escenarios pertenezcan al mismo equipo de diseñadores. Se pide: Demostrar que el problema es NP-Completo. (HINT!: Tal vez le resulte útil clique)

Fuerza Bruta

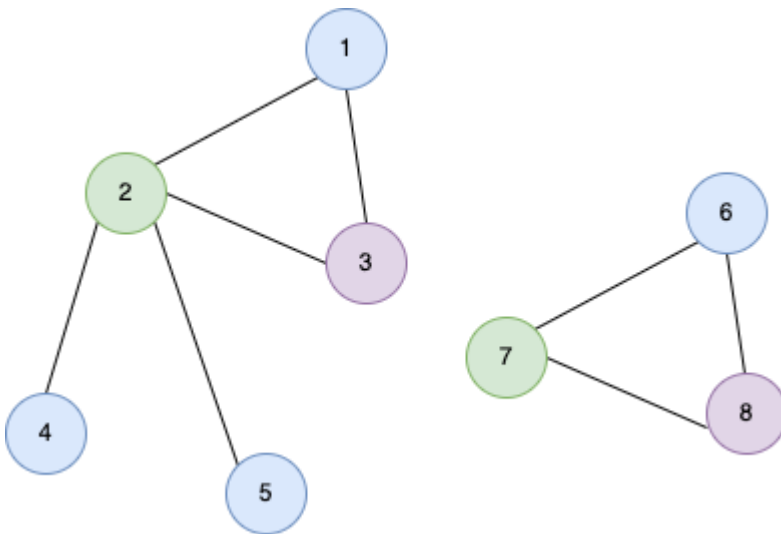
Explicación de la solución

Para solucionar el problema con la técnica de Backtracking se plantea partir de un nodo inicial (cualquiera), iterar por cada uno de los colores K disponibles, pintar el nodo de dicho color y sobre cada iteración, realizar el mismo procedimiento sobre los nodos adyacentes aún no coloreados. Para visitar los nodos adyacentes se utiliza la técnica DFS de recorrido de grafos.

En cada paso también se valida que el color elegido para el nodo actual sea válido (ningún nodo adyacente ya tenga seleccionado dicho color), y en caso que no lo sea, se procede a probar con otro color. En caso que todos los colores hayan sido probados para un nodo particular y la solución siga sin ser válida, se retorna que la solución no es válida.

Este procedimiento se ejecuta mientras no hayan nodos sin haber sido pintados, dado que el grafo puede contener más de una componente no conexa, y se debe ejecutar el algoritmo de Backtracking para cada una de ellas.

Nota: otra solución válida consiste en recorrer los nodos del grafo sin iterar sobre sus adyacentes como en la solución propuesta, si no uno a uno y en cada paso, a la hora de pintarlos de un color, verificar si el mismo puede ser pintado de dicho color o no (siempre y cuando el color que tengan sus adyacentes no sea el mismo que el color que se plantea pintar)



En el ejemplo planteado, se comenzará recorriendo el grafo por el nodo 1 de la componente conexa izquierda.

- 1) El mismo se intentará pintar de color **celeste**. Al no estar pintados sus adyacentes, se considera el color como válido y se procede a iterar sus adyacentes
- 2) Se toma el nodo 2. Se intenta pintar del color **celeste**, pero al validarse sus adyacentes sin pintar, el nodo 1 ya lo tiene, por lo que se pinta con el siguiente color: **verde**. Al ser válido el mismo, se procede a iterar sus adyacentes
- 3) Se toma el nodo 4. Se pinta con el color **celeste**, al ser válido y al no tener nodos adyacentes sin pintar, se devuelve True como solución válida y se vuelve a la iteración del nodo 2
- 4) Se toma el nodo 5 como segundo adyacente del nodo 2. Se pinta con el color **celeste**, al ser válido y al no tener nodos adyacentes sin pintar, se devuelve True como solución válida
- 5) Se toma el nodo 3 como tercer adyacente del nodo 2. Se intenta pintar del color **celeste**, pero no es válido pues el nodo 1 posee dicho color, de igual manera con el siguiente color verde dado que el nodo adyacente 2 lo posee. Se pinta con el tercer color: violeta. El mismo es válido. Se devuelve True como solución dado que no existen nodos adyacentes sin pintar
- 6) Dado que el nodo 2 y nodo 1 no poseen otros nodos adyacentes sin pintar, se retorna como válida la solución de la primer componente conexa
- 7) Se procede con la siguiente componente conexa y se toma un nodo cualquiera, en este caso el nodo 6. Se pinta de color celeste y se iteran sus adyances
- 8) Se toma el nodo 7, se pinta de color verde al no poder pintarse de color celeste dado que el nodo 1 lo posee.
- 9) Se toma el nodo 8 como adyacente del nodo 7 y se pinta del tercer color violeta al no estar disponibles los colores celeste y verde. Luego se devuelve como válida la solución de la segunda componente conexa y finaliza la ejecución del algoritmo

Pseudocódigo

```
Backtracking (nodo, coloreo)
    Por cada color en Colores
        asigno color a nodo

        Por cada adyacente en nodo.adyacentes
            # valido que el color del adyacente sea distinto a color. Si es igual, continuo
            con otro color

        Por cada adyacente en nodo.adyacentes
            Si adyacente no fue pintando
                es_valido, nuevo_coloreo = Backtracking (adyacente, coloreo)

                si no es válido, continuo con otro color
                si es válido, continuo con otro adyacente usando nuevo_coloreo

            # todos los adyacentes fueron pintados
            Return True

# ningún color fue válido para pintar al nodo
Return False

Mientras queden nodos sin pintar:
    # ya que el grafo puede tener componentes no conexas, se ejecuta mientras queden nodos sin
    pintar
    nodo = elijo un nodo sin pintar
    es_valido, coloreo = Backtracking (nodo, coloreo)

    si no es valido, devuelvo False # no se puede colorear el grafo

Return coloreo
```

Análisis de complejidad

La complejidad temporal del algoritmo planteado es $O(N * K^N)$, donde K representa la cantidad de colores y N la cantidad de nodos que tiene el grafo. En el peor de los casos, se iteran todas las combinaciones de colores por cada nodo del grafo (K^N), pero además por cada iteración se recorren los adyacentes del nodo asignado, donde en el peor de los casos es N , por lo que la complejidad final es $O(N * K^N)$. Las demás operaciones que se realizan dentro de cada iteración, tales como la asignación de color se realizan en $O(1)$ y no afectan la complejidad de la solución.

La complejidad espacial del algoritmo planteado es $O(N)$, dado que la variable coloreo puede contener N valores y es la única variables que se utiliza para almacenar los valores de color que cada nodo tiene asignado

Greedy

Pseudocódigo

Ordenamos los faros por kilómetro de inicio de cobertura Si de forma creciente

```
ultimo_km_cubierto = 0
```

```
faros_elegidos = []
```

```
proximo_faro_optimo_a_elegir = {vacio}
```

Por cada faro en faros_ordenados:

```
Si faro.Si <= ultimo_km_cubierto:
```

```
    # puedo considerar al faro para cubrir el siguiente tramo
```

```
Si faro.Ei > proximo_faro_optimo_a_elegir.Ei:
```

```
    # el faro actual cubre la mayor distancia
```

```
    proximo_faro_optimo_a_elegir = faro
```

```
Si no:
```

```
    # el faro actual no cubre el punto ultimo_km_cubierto, lo que significa que si existe  
    un proximo_faro_optimo_a_elegir que lo cubra, ese será el que mayor distancia cubra.
```

```
Si proximo_faro_optimo_a_elegir <> {vacío}:
```

```
    faros_elegidos.add(proximo_faro_optimo_a_elegir)
```

```
    ultimo_km_cubierto = proximo_faro_optimo_a_elegir.Ei # Partimos del último
```

```
kilómetro cubierto
```

```
Si faro.Si <= ultimo_km_cubierto AND faro.Ei > ultimo_km_cubierto:
```

```
    proximo_faro_optimo_a_elegir = faro
```

```
Si no:
```

```
    proximo_faro_optimo_a_elegir = {vacío}
```

```
# Falta seleccionar el último faro pendiente a elegir
```

```
Si proximo_faro_optimo_a_elegir <> {vacío}
```

```
    faros_elegidos.add(proximo_faro_optimo_a_elegir)
```

```
# Verificamos que el último faro elegido cubre el final de la costa
```

```
Si faros_elegidos.pop().Ei < ultimo_km_de_costa:
```

```
    Return "no existe solución válida"
```

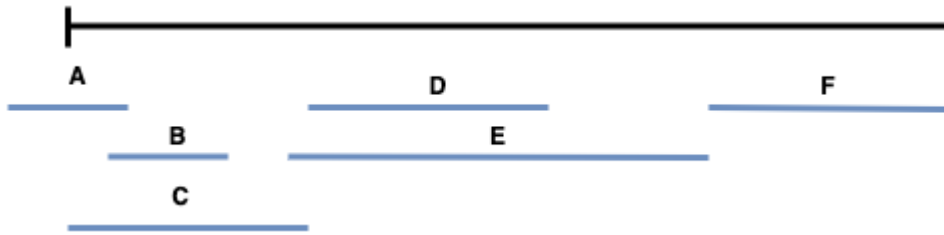
```
Return faros_elegidos
```

Explicación de la solución

Para solucionar el problema se plantea ordenar inicialmente los faros por orden de inicio de cobertura de forma creciente y luego iterar los mismos según el orden establecido. La idea es, al recorrerlos, seleccionar el faro que mayor kilómetros abarca, siempre considerando desde un punto origen hasta la un punto destino Ei. Inicialmente el punto origen será 0 (el inicio de la costa) y se seleccionará el faro que cubra el punto 0 (Si ≤ 0) y que mayor distancia ocupe (mayor Ei). Dado dicho escenario inicial, podemos ver que el mismo es óptimo pues se selecciona el faro que cubre mayor distancia en la costa y además cubre el punto inicial de la costa. Si extendemos este concepto para una próxima iteración, el nuevo punto de origen será el punto de finalización (Ei) del faro seleccionado en la iteración anterior, por lo que la nueva iteración seleccionará un nuevo faro que cubra el punto Ei de origen y cuyo punto de finalización Ei sea máximo. Extendiendo por inducción este concepto para los demás faros restantes hasta N nos otorga la solución óptima del problema.

El algoritmo también considera los casos donde la solución no existe, cuando quedan tramos de costa sin cubrir, ya sea al comienzo, en algún tramo intermedio o al final.

Ejemplo



Basado en el siguiente ejemplo, el algoritmo ordenará los faros según su punto de kilómetro inicial de cobertura Si

A, C, B, E, D, F

Luego comenzará las iteraciones

- 1) faro = A
ultimo_km_cubierto = 0
proximo_faro_optimo_a_elegir = A
faros_elegidos = []
- 2) faro = C
ultimo_km_cubierto = 0
proximo_faro_optimo_a_elegir = C # ya que cubre más kilómetros que A y también cubre al punto 0
faros_elegidos = []
- 3) faro = B
ultimo_km_cubierto = C.Ei # actualizamos ya que B.Si > ultimo_km_cubierto
proximo_faro_optimo_a_elegir = {vacío}
faros_elegidos = [C]
- 4) faro = E
ultimo_km_cubierto = C.Ei
proximo_faro_optimo_a_elegir = E
faros_elegidos = [C]
- 5) faro = D
ultimo_km_cubierto = C.Ei
proximo_faro_optimo_a_elegir = E
faros_elegidos = [C]
- 6) faro = F
ultimo_km_cubierto = E.Ei # actualizamos ya que F.Si > ultimo_km_cubierto
proximo_faro_optimo_a_elegir = F
faros_elegidos = [C, E]
- 7) faros_elegidos = [C, E, F] # Solución al problema

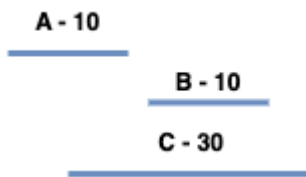
Análisis de complejidad

La complejidad temporal de la solución es $O(N \log(N))$, N representando la cantidad de faros, ya que inicialmente se ordenan los mismos por cobertura inicial Si, lo cual posee una complejidad de $O(N \log(N))$. La siguiente iteración se realiza en $O(N)$ dado que en el peor de los casos se itera cada uno de los faros y dentro de la iteración solo se realizan operaciones $O(1)$ (insertar un faro en la lista de faros_elegidos, asignar el nuevo faro óptimo a elegir)

La complejidad espacial de la solución es $O(N)$ dada por el arreglo faros_elegidos, donde en el peor de los casos los N faros formarán parte del mismo.

Programación dinámica

Contraejemplo



Como contraejemplo podemos ver que en el caso que se elijan las propuestas que duren menos, se elegirán las propuesta A y B, cuya duración es la menor y no se encuentran superpuestas, mientras que no la C al tener mayor duración y superponerse con A y B no será elegida. La solución óptima es elegir la propuesta C ya que otorga una ganancia de 30, contra la suma de las ganancias de A y B que suman 20.

Ecuación de recurrencia

Defino $Opt[i]$ como la mayor ganancia obtenida hasta la propuesta i

Defino el vector P donde cada posición (i) del vector corresponde con la última propuesta (j) que no se solape con la propuesta i .

Considero las propuestas ordenadas según su finalización

```
v[i] si i = 1 # caso base
Opt[i] =
    max { v[i] + Opt[ P[i] ], Opt[i - 1] }
```

Pseudocódigo

Ordeno el vector V de propuestas por finalización de forma creciente

Sea P el vector definido que almacena la última propuesta sin solapamiento a cada propuesta i

$Opt[1] = V[1]$ # caso base

Desde $i=2$ hasta N # N cantidad de propuestas

Si $(v[i] + Opt[P[i]]) > Opt[i - 1]$

$Opt[i] = v[i] + Opt[P[i]]$

Si no

$Opt[i] = Opt[i - 1]$

Return $Opt[N]$

Para reconstruir las solución se plantea el siguiente algoritmo donde se itera desde la última propuesta hasta la primera, eligiendo en cada paso si la propuesta fue elegida o no (lo cual se determina según el óptimo de la solución actual sea distinto al óptimo del ejemplo anterior. Si es igual, significa que la solución óptima actual consiste en considerar el elemento anterior y no el actual, si es distinto, se debe considerar el elemento anterior compatible $P[i]$)

proposiciones_elegidas = []

$i = N$ # se parte de la propuesta final

while $i > 0$:

 Si $(Opt[i] <> Opt[i - 1])$

 proposiciones_elegidas.add(i)

 Si $P[i] = \{\text{vacío}\}$:

 break

$i = P[i]$

$i -= 1$

Return propuestas_elegidas

Explicación de la solución

Para resolver el problema se plantea ordenar las propuestas por tiempo de finalización de forma creciente con el objetivo de luego iterar las mismas por ese orden y determinar en cada paso cuál es la máxima ganancia que se puede obtener al haber iterado hasta una propuesta particular i . Para determinar dicho valor se considera seleccionar la ganancia actual " i ", sumado a la ganancia óptima de la última propuesta posible de elegir (la cual no presenta un solapamiento con la actual), o simplemente elegir la ganancia óptima de la propuesta anterior (esta podrá elegirse ya que al no elegir la propuesta actual, no presentará un solapamiento). Partiendo del caso base donde el óptimo representa elegir realizar primer propuesta (dado que no presenta hasta el momento solapamiento con otras), se extiende la solución hasta analizar el óptimo de la última propuesta N elegida.

Análisis de complejidad

La complejidad temporal de la solución es $O(N \log(N))$, donde N representa la cantidad de propuestas. Esta se da por ordenar el vector V de propuestas por tiempo de finalización.

Las demás operaciones incluyen iterar las N propuestas para definir el $Opt[i]$ en cada paso, lo cual incluye una complejidad de $O(N)$ y crear el vector P con las últimas propuestas sin solapamiento para cada propuesta i . La complejidad de la creación de dicho vector es $O(N)$.

La complejidad de reconstruir la solución es $O(N)$ dado que en el peor de los casos se iteran todas las propuestas.

La complejidad espacial de la solución es $O(N)$, dado por el vector Opt , que al finalizar la ejecución del algoritmo contendrá N valores. En el caso del algoritmo de reconstrucción, en el peor de los casos el vector `propuestas_elegidas` contiene las N propuestas y por lo tanto la complejidad sigue siendo $O(N)$.

Redes de Flujo

Pseudocódigo

```
ford_fulkerson (G)
    Inicialmente el flujo  $f(e) = 0$  para todo eje  $e$  en el grafo  $G$ 
    Inicializar  $Gr$  grafo residual según  $G$  y flujo  $s$ - $t$   $f = 0$ 
    Mientras haya un camino  $s$ - $t$  en  $Gr$ 
        Sea  $P$  un camino  $s$ - $t$  en  $Gr$ 
        Sea  $w$  el valor de flujo del cuello de botella del camino  $P$ 

        Actualizar el flujo  $w$  en el camino  $P$  en  $G$ 
        Actualizar  $Gr$ 
    Retornar  $f$ 

asignar_camaras (G, M)
    flujo_maximo = ford_fulkerson(G)
    if flujo_maximo < M:
        return "no es posible asignar las cámaras a las piezas"

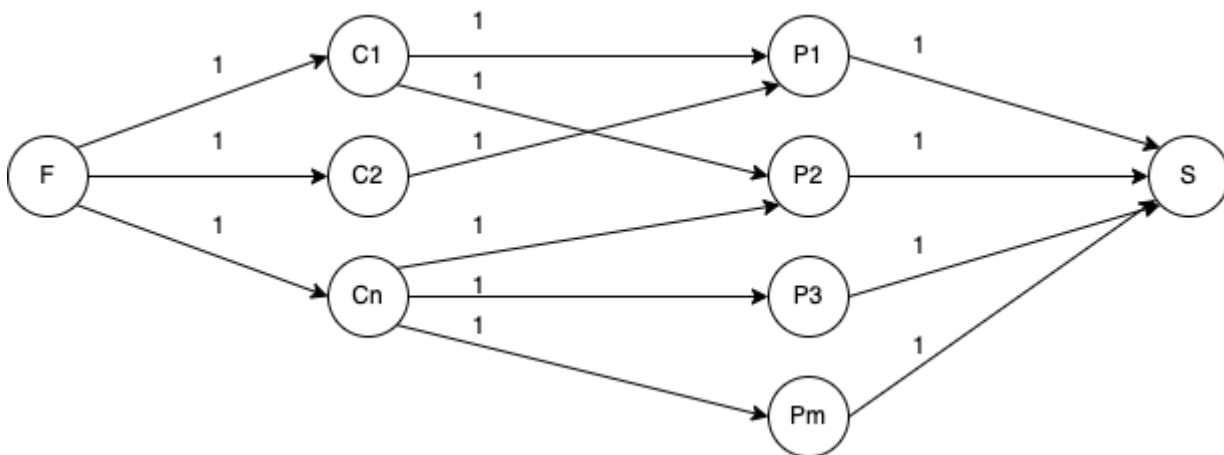
    Recorrer las aristas que van desde las cámaras hacia las piezas y seleccionar aquellas que
    poseen flujo 1

    Return asignación

armar grafo creando cámaras y piezas como nodos, uniendo la fuente a las cámaras con aristas de
capacidad 1, las cámaras a las piezas disponibles con aristas de capacidad 1 y por último las piezas
al sumidero con aristas de capacidad 1. # Armado de la red de flujo

return asignar_camaras (G, M) # Sea M la cantidad de cámaras
```

Explicación de la solución



Para resolver el problema se plantea una red de flujo donde cada cámara está representada por un nodo, a su vez, cada pieza está representada también por un nodo. La Fuente se encuentra unida a cada cámara con capacidad 1 (representando que cada cámara solo se puede seleccionar 1 vez para cubrir una pieza). Las cámaras se encuentran unidas a las piezas disponibles para cubrir, las mismas también se encuentran unidas con capacidad 1 representando pueden efectivamente cubrir dicha pieza. Las piezas se encuentran unidas al Sumidero con capacidad 1 representando que las piezas solo pueden estar cubiertas por solo 1 cámara.

Luego se aplica el algoritmo de Ford-Fulkerson para obtener el flujo máximo.

Si el flujo máximo es menor que la cantidad de piezas, representa que el problema no puede ser resuelto dado que no alcanzan las cámaras disponibles para cubrir las piezas.

En caso de que el flujo máximo sea igual a la cantidad, se iteran aquellas aristas entre las cámaras y las piezas, y aquellas que el flujo sea positivo (1) se seleccionan. Luego se retorna dicha selección.

Análisis de complejidad

La complejidad temporal de la solución es $O(N * M^2)$. La misma está dada por la complejidad de la ejecución de Ford-Fulkerson $O(|E| * C)$, donde C puede acotarse a M ya que el flujo máximo será como máximo al cantidad de piezas elegidas. La cantidad de aristas es en el peor de los casos igual a $N*M + N + M$. Por lo tanto $O(|E| * C) = O((N*M + N + M) * M) = O(N*M^2 + N*M + M^2) = O(N * M^2)$.

La complejidad de asignar las cámaras con las piezas corresponde a la complejidad $O(E) = O(N*M)$ dado que se iteran las aristas que van desde las cámaras hacia las piezas. Dicha complejidad no influye en la complejidad final.

La complejidad de armado de la red de flujo corresponde a formar los nodos $O(N + M + 2) = O(N + M)$ y las aristas $O(N*M + N + M) = O(N*M)$, por lo que tampoco influye en la complejidad final

La complejidad espacial de la solución corresponde a la cantidad de nodos y aristas del problema que representan el grafo a utilizar. $O(V + E) = O(N*M)$

Clases de complejidad

Explicación de la solución

Para demostrar que el problema es NPC, procedemos a demostrar que el mismo pertenece a NP y NPH.

Llamaremos AV al problema mencionado de "Aventura gráfica"

Demostración de que el problema es NP

Planteamos un certificador del problema

```
certificador (equipos, escenarios, asignaciones, equipo_elegido, K)
```

```
Validar que |asignaciones| = K
Validar que equipo_elegido existe en equipos

escenarios_elegidos = []
Por cada asignacion en asignaciones:
    equipo, escenario en asignacion

    Si equipo no es igual a equipo_elegido:
        return "no es válida la asignación"
    Si escenario no se encuentra en escenarios:
        return "no es válida la asignación"
    Si equipo no propuso el escenario
        return "no es válida la asignación"
    Si escenario no es compatible con escenarios_elegidos
        return "no es válida la asignación"

    escenarios_elegidos.add(escenario)

Return "solución válida"
```

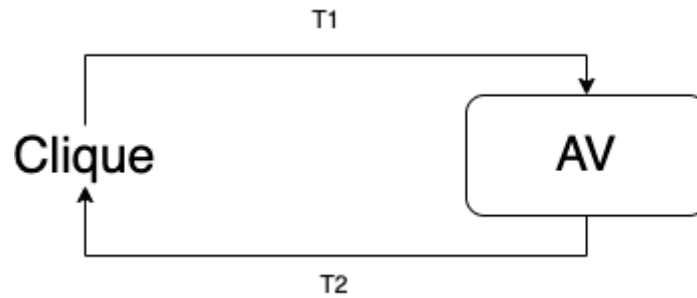
La complejidad del certificador es $O(K*N)$, donde K representa la cantidad de asignaciones y N la cantidad de escenarios. Las operaciones planteadas en el certificador son las siguientes:

- Validar que las asignaciones sean K puede realizarse en $O(1)$ en caso que la estructura utilizada guarde esa información (por ejemplo una lista)
- Verificar que el equipo exista $O(1)$
- Iterar las K asignaciones, donde en cada iteración se valida que el equipo haya propuesto el escenario $O(1)$, si el escenario es compatible con escenarios_elegidos (lo cual puede verificarse recorriendo cada escenario dentro de la lista escenarios_elegidos) $O(N)$ y validar que los escenarios existan $O(N)$

Dado que la complejidad del certificador es Polinomial, podemos afirmar que el problema es NP

Demostración de que el problema es NPH

Para demostrar que el problema es NPH, procedemos a plantear una reducción al mismo. Utilizamos el problema de Clique para hacerlo.



Transformación de una instancia de problema de Clique a una instancia del problema de AV

Se crea un equipo X en el problema de AV

Por cada nodo en la instancia del problema de Clique, se crea un escenario distinto en el problema de AV y se asigna dicho escenario al equipo X

Por cada arista (U, V) en la instancia del problema de Clique, se determina que el escenario asignado del nodo U sea compatible con el asignado del nodo V

Sea K, el tamaño del clique en el problema de Clique, se define el mismo valor K para representar la cantidad de escenarios compatibles en el problema AV

La complejidad de la transformación es $O(V + E)$ dado que se necesitan iterar todos los nodos para definir qué escenarios serán utilizados y todas las aristas para determinar la compatibilidad entre escenarios

Dicha transformación va a permitir que al ejecutar el algoritmo de AV sobre la nueva instancia creada, se encuentren, en caso que sea posible, K escenarios que sean compatibles entre sí. Si dichos escenarios son compatibles entre sí, también representa que en el problema de Clique los mismos asociados a nodos poseen aristas en común, por lo tanto, los mismos forman un clique. Al solo asignar los escenarios a 1 equipo, permite que ninguna de las aristas/escenarios no sea considerada. En caso de no encontrar una compatibilidad de escenarios al resolver AV, significa que no se encontrará un clique en el problema de Clique.

Transformación del resultado del problema de AV al resultado de Clique

Para obtener el resultado del problema de Clique se debe realizar la transformación de cada escenario incluido en las asignaciones del problema de AV en el nodo del grafo original del problema de Clique. Por lo explicado, dichos nodos serán parte del clique de tamaño K buscado.

La complejidad de la transformación es $O(V)$ dado que se necesitan iterar todos los escenarios para definir qué nodos pertenecen al clique solución.

De esta manera podemos justificar que el problema de Clique $\leq$ AV, y por lo tanto justificar que AV es al menos tan difícil que Clique, por lo que es considerado NPH.

Al ser demostrado que AV es NP y NPH, el mismo es considerado NPC.